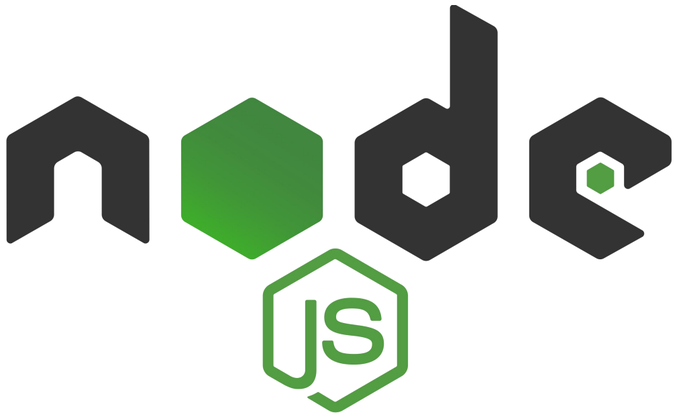
iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

CAPACITAÇÃO EM DESENVOLVIMENTO FULL STACK

Módulo 05: Desenvolvimento Back-end com Node.js e Express
Instrutor: Janei Vieira
Aula 5: Bancos de Dados SQL/NoSQL



iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Janei Vieira

 Full Stack Developer 🧑‍💻

🎉 24 anos de experiência em tecnologia. Especialista em infraestrutura, automação, containers (Docker/Kubernetes) e desenvolvimento back-end para sistemas comerciais e aplicações em .NET C#, Python e Node.JS

🧑‍💻 Foco em Front-ends com Razor Pages e Streamlit.



Histórico Acadêmico

- **Graduação:** Gestão da Tecnologia da Informação.
- **MBA:** Em Gestão Ágil de Projetos e Processos
- **Especializações:**
  - Redes e Computação Distribuída (**IFMT**).
  - Engenharia DEVOPS (**IFMT**)
  - Big Data (Ciência de Dados).
  - Computação Em Nuvem (Cloud Computing).

Mestrando: Propriedade Intelectual para Transferência de Tecnologia e inovação (**UFRR**)

https://lattes.cnpq.br/4083863936027227

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

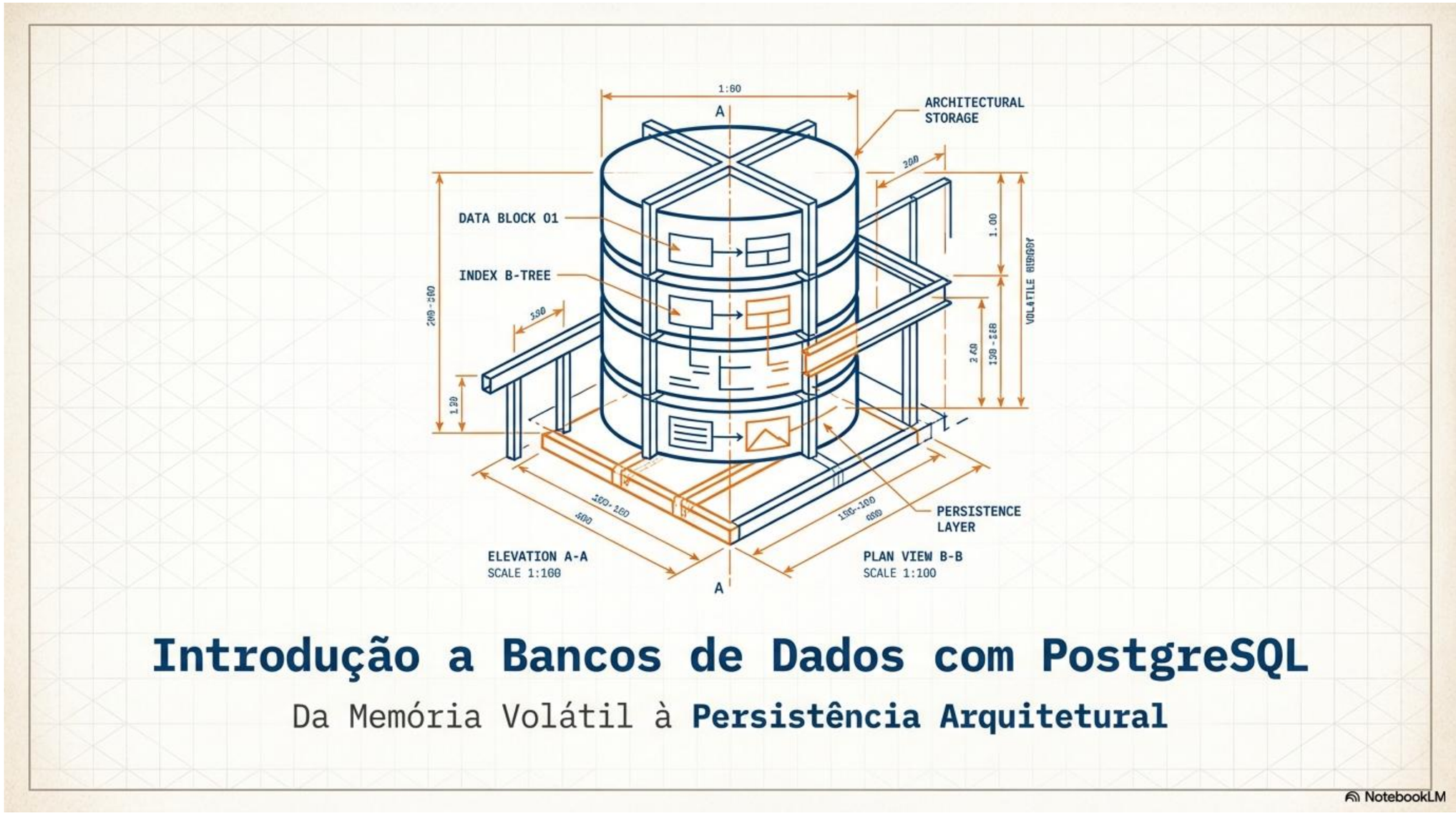
Objetivos da Aula

- **Compreender** o conceito de SQL x NoSQL
- **Entender** As diferenças entre PostgreSQL e MongoDB.
- **Criar** Um Banco de Dados e Algumas tabelas usando SQL
- **Aprender** Como realizar transações **CRUD**

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

Revisão - O que vimos até aqui?

- **Conceito:** Função (**req**, **res**, **next**) entre request e resposta — next() libera
- **Cadeia:** Ordem importa! logger → express.json() → rota final
- **express.json():** Transforma req.body JSON em objeto utilizável
- **Logging**: console.log(method + url + timestamp) em cada request



Introdução a Bancos de Dados com PostgreSQL

Da Memória Volátil à **Persistência Arquitetural**

O Fim da Amnésia Digital



Armazenamento em Memória (RAM)

⚠ - Rápido, porém efêmero.
⚠ - Dados sobrevivem apenas durante a execução do processo.



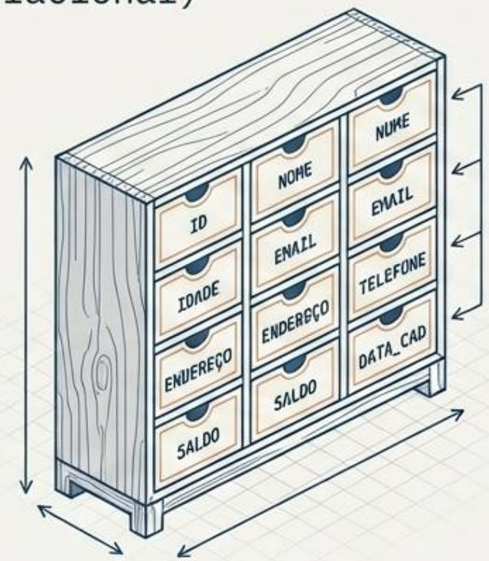
Persistência em Banco de Dados

✓ - Armazenamento em disco gerenciado por um SGBD.
✓ - Dados sobrevivem a reinicializações, falhas e atualizações.
✓ - Permite consultas complexas e segurança transacional.

NotebookLM

O Grande Divisor: SQL vs. NoSQL

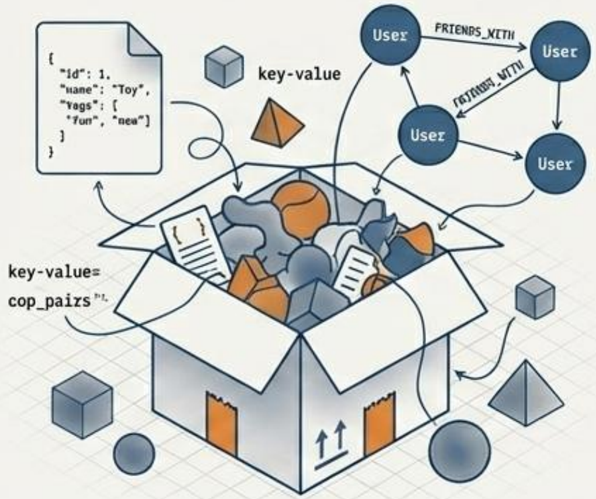
SQL (Relacional)



Um guarda-roupa com divisórias fixas. Esquema rígido e predefinido. Ideal para consistência e transações seguras.

PostgreSQL | MySQL | SQL Server

NoSQL (Não Relacional)



Uma caixa de brinquedos. Estrutura livre e dinâmica (documentos, grafos). Ideal para alta escalabilidade e evolução rápida.

MongoDB | Redis | Neo4j

NotebookLM

Matriz Diagnóstica: Relacional vs. Não Relacional

Característica	SQL (PostgreSQL)	NoSQL (MongoDB)
Estrutura	 Tabelas, linhas, colunas (Rígido)	 Coleções, documentos BSON/JSON (Flexível)
Escalabilidade	 Vertical (Aumenta a potência do servidor)	 Horizontal (Distribui em vários servidores)
Consistência	Alta (Garantia de transações ACID)	Eventual (Foco em disponibilidade/performance)
Exemplo de Uso	 Gestão de tarefas, dados financeiros	 Feed de notícias, Big Data

NotebookLM

O Padrão Ouro Relacional: PostgreSQL



Especificações de Materiais: Numéricos e Lógicos

Numéricos

`integer` `(4 bytes)`
Escolha padrão para números inteiros.

`numeric(p, s) / decimal`
Precisão exata sem limite.

`real / double precision`
Ponto flutuante, precisão inexata.

`serial`
Inteiro com autoincremento (ideal para IDs).

Monetário

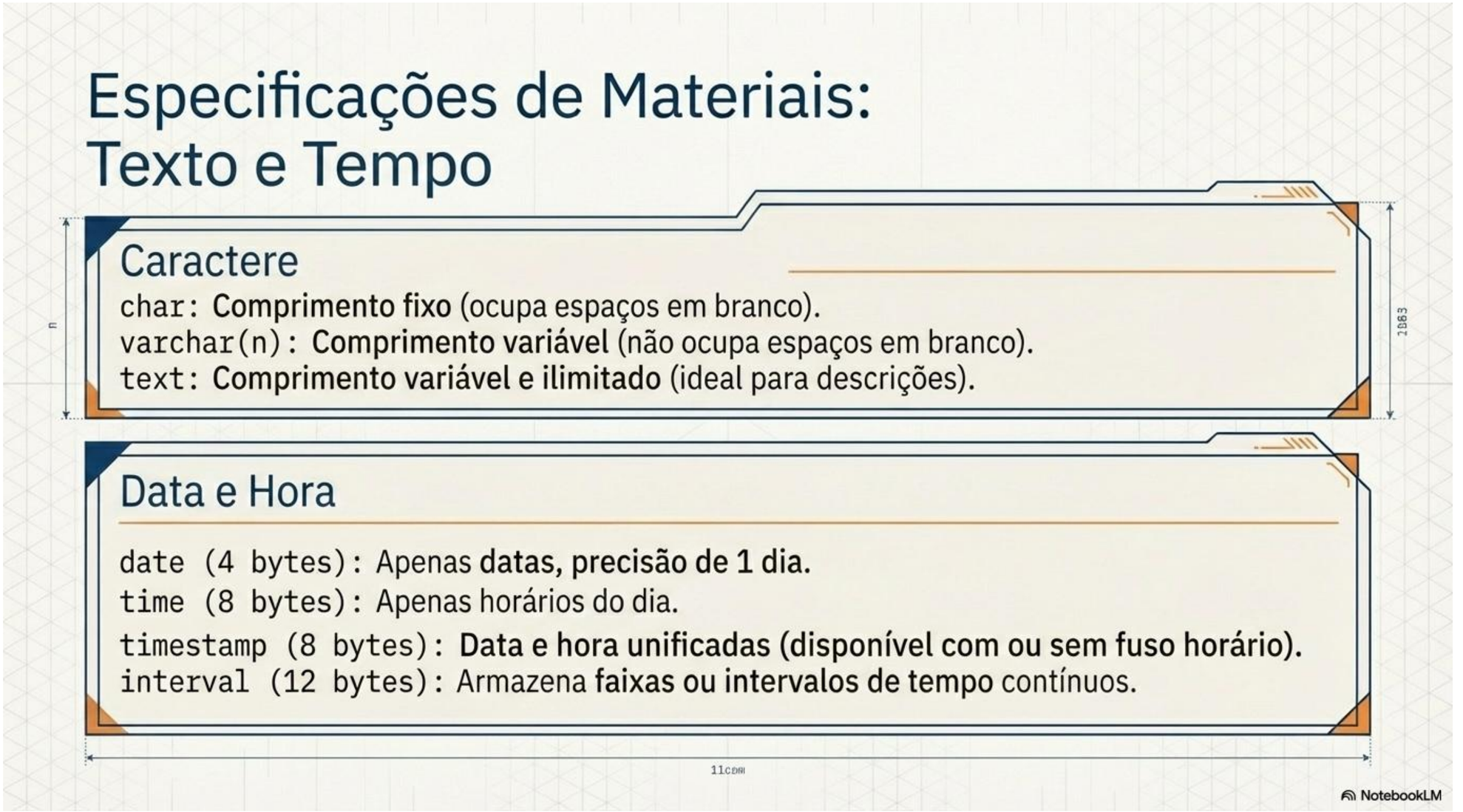
`money`
Armazena valores de moeda com segurança.
Faixa: -21474836.48 a +21474836.47

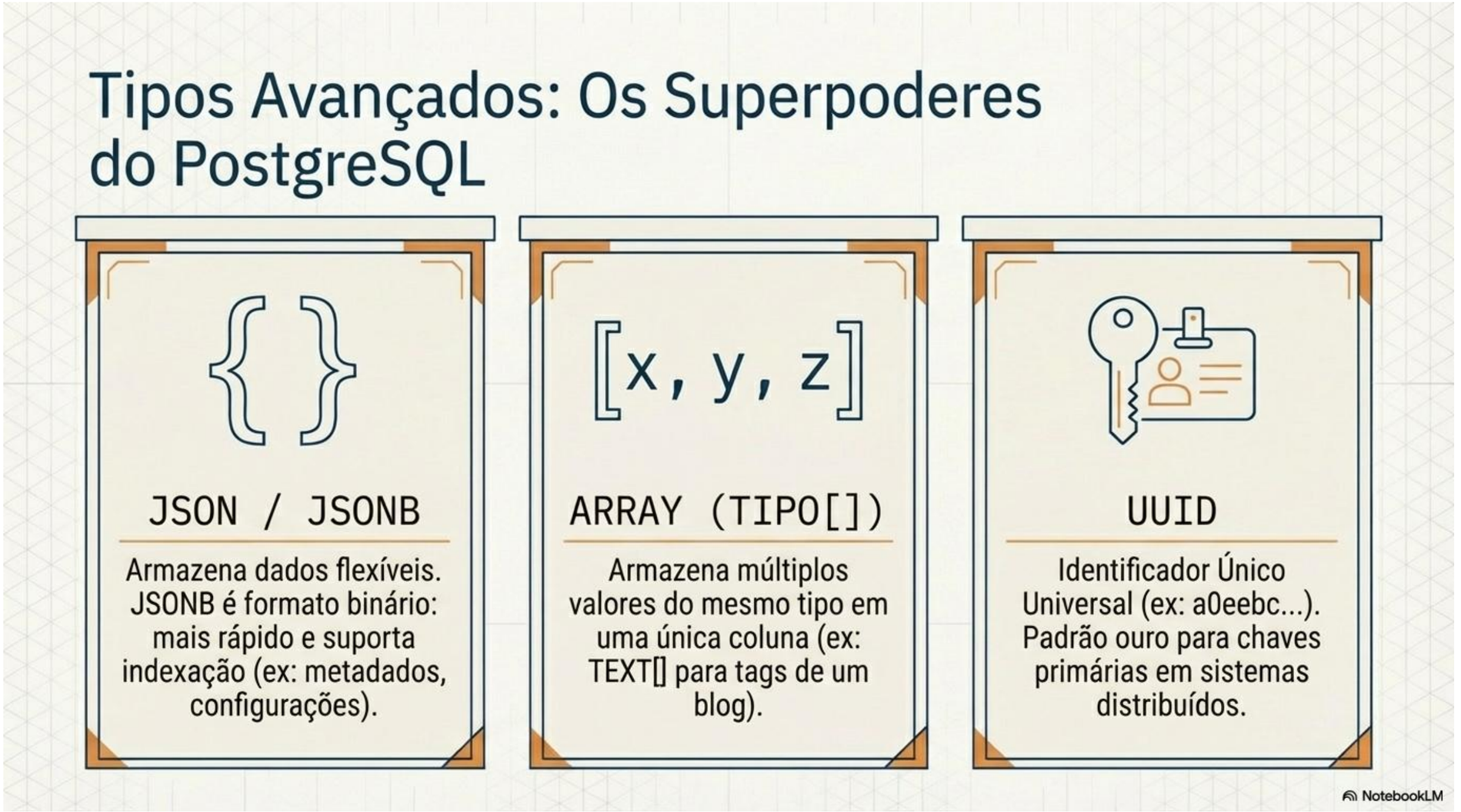
Lógicos

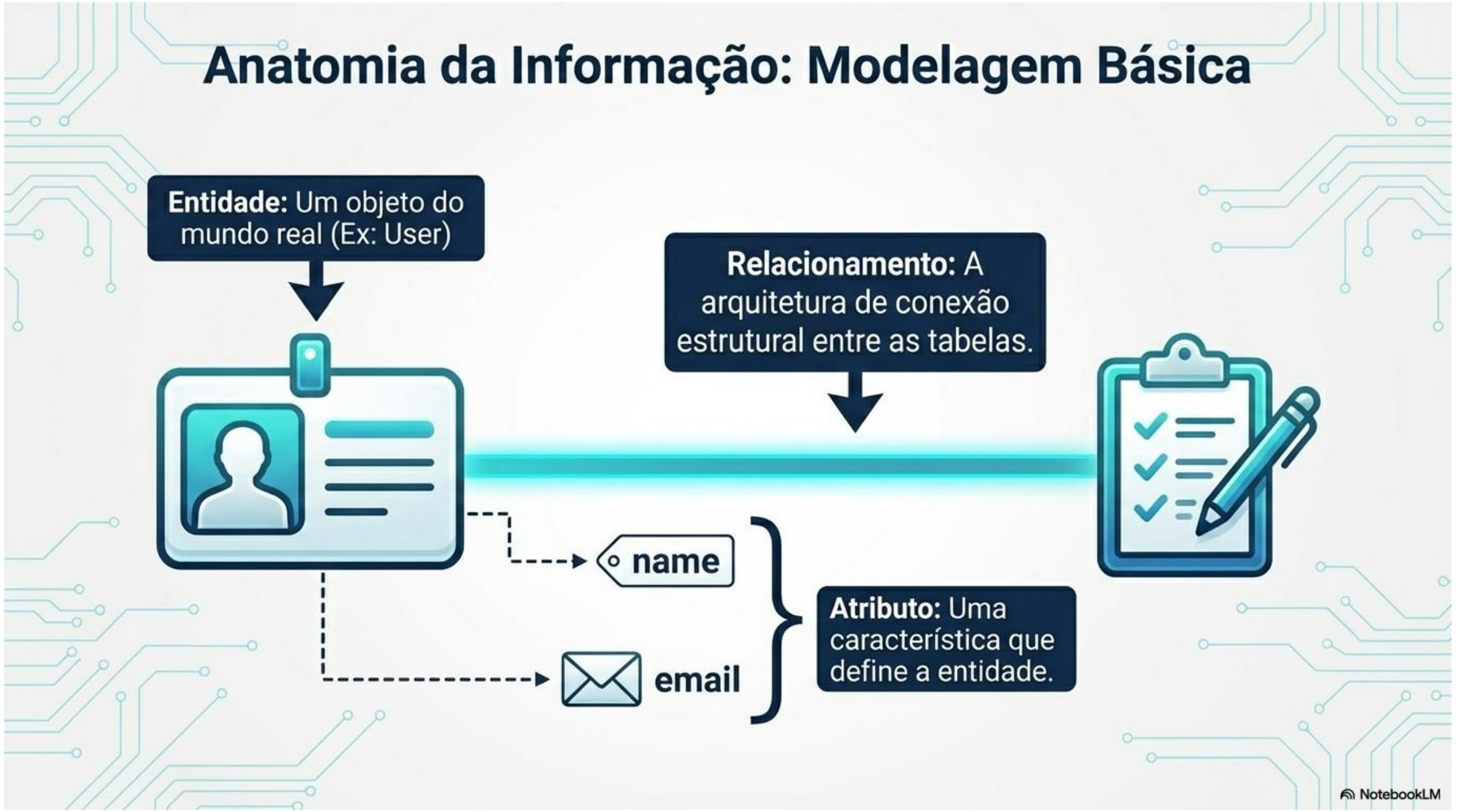
`boolean`
Aceita estado binário.

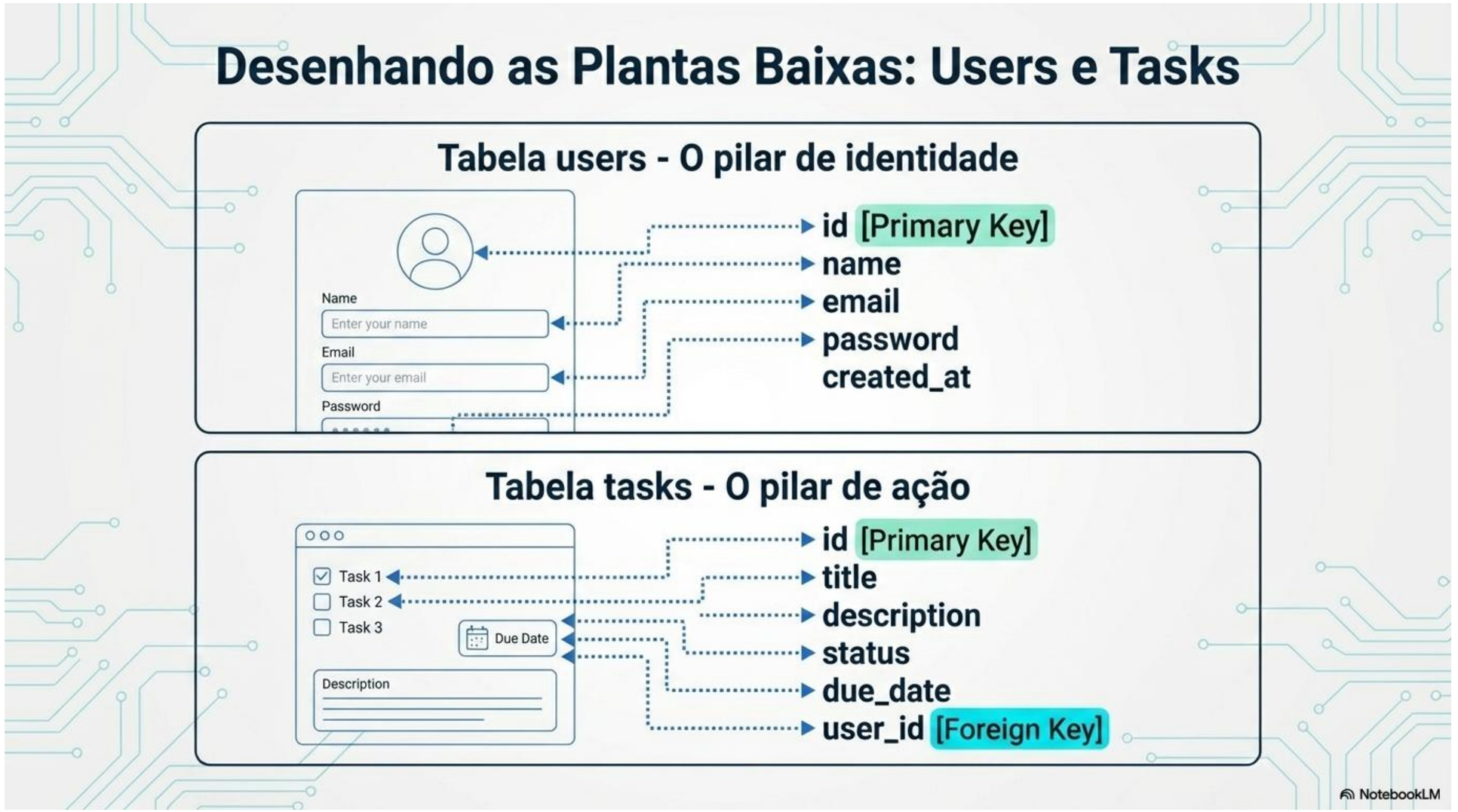
`TRUE ('t', 'true', 'y', '1')`
`FALSE ('f', 'false', 'n', '0')`

NotebookLM


Especificações de Materiais:
Texto e Tempo
Caractere
char: Comprimento fixo (ocupa espaços em branco).
varchar(n): Comprimento variável (não ocupa espaços em branco).
text: Comprimento variável e ilimitado (ideal para descrições).
Data e Hora
date (4 bytes): Apenas datas, precisão de 1 dia.
time (8 bytes): Apenas horários do dia.
timestamp (8 bytes): Data e hora unificadas (disponível com ou sem fuso horário).
interval (12 bytes): Armazena faixas ou intervalos de tempo contínuos.
NotebookLM


Tipos Avançados: Os Superpoderes do PostgreSQL
{ }
JSON / JSONB
Armazena dados flexíveis. JSONB é formato binário: mais rápido e suporta indexação (ex: metadados, configurações).
[x, y, z]
ARRAY (TIPO[])
Armazena múltiplos valores do mesmo tipo em uma única coluna (ex: TEXT[] para tags de um blog).
UUID
Identificador Único Universal (ex: a0eebc...). Padrão ouro para chaves primárias em sistemas distribuídos.
NotebookLM


Anatomia da Informação: Modelagem Básica
Entidade: Um objeto do mundo real (Ex: User)
Relacionamento: A arquitetura de conexão estrutural entre as tabelas.
name
email
Atributo: Uma característica que define a entidade.
NotebookLM


Desenhando as Plantas Baixas: Users e Tasks
Tabela users - O pilar de identidade
Name
Enter your name
Email
Enter your email
Password
id [Primary Key]
name
email
password
created_at
Tabela tasks - O pilar de ação
Task 1
Task 2
Task 3
Due Date
Description
id [Primary Key]
title
description
status
due_date
user_id [Foreign Key]
NotebookLM

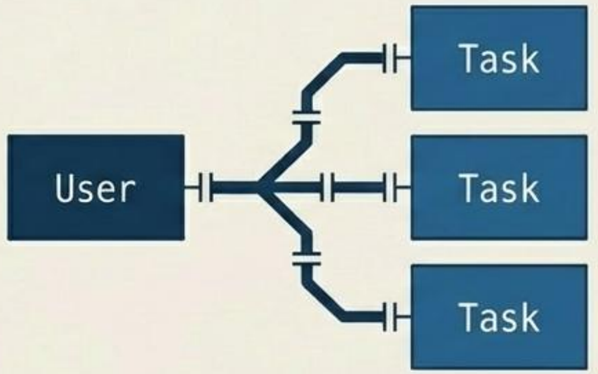
Fundamentos da Modelagem Relacional

Entidade: Um objeto do mundo real (ex: `User`, `Task`). Representado como caixas sólidas.

Atributo: Características da entidade (ex: `name`, `status`). Representado como linhas de dados dentro da caixa.

Relacionamento: A conexão estrutural geométrica entre as entidades.

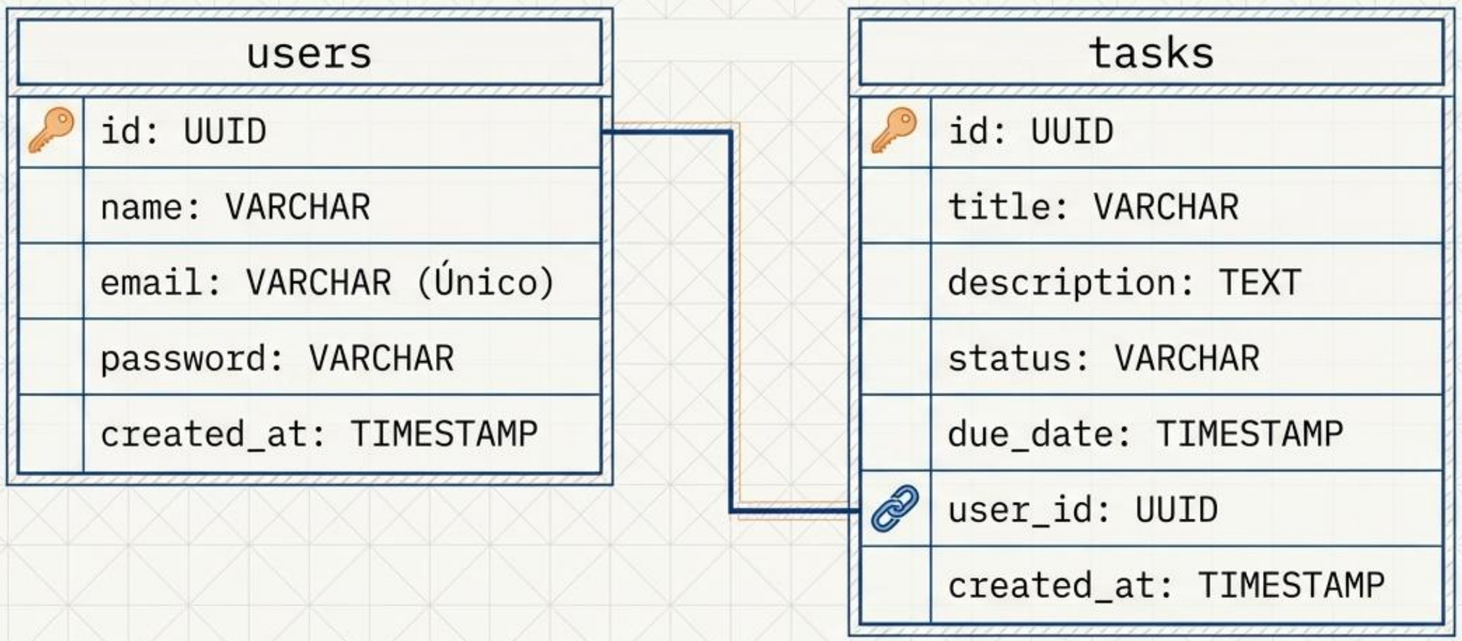
O Relacionamento 1-N



Um para Muitos: Um usuário possui várias tarefas, mas cada tarefa pertence a apenas um usuário.

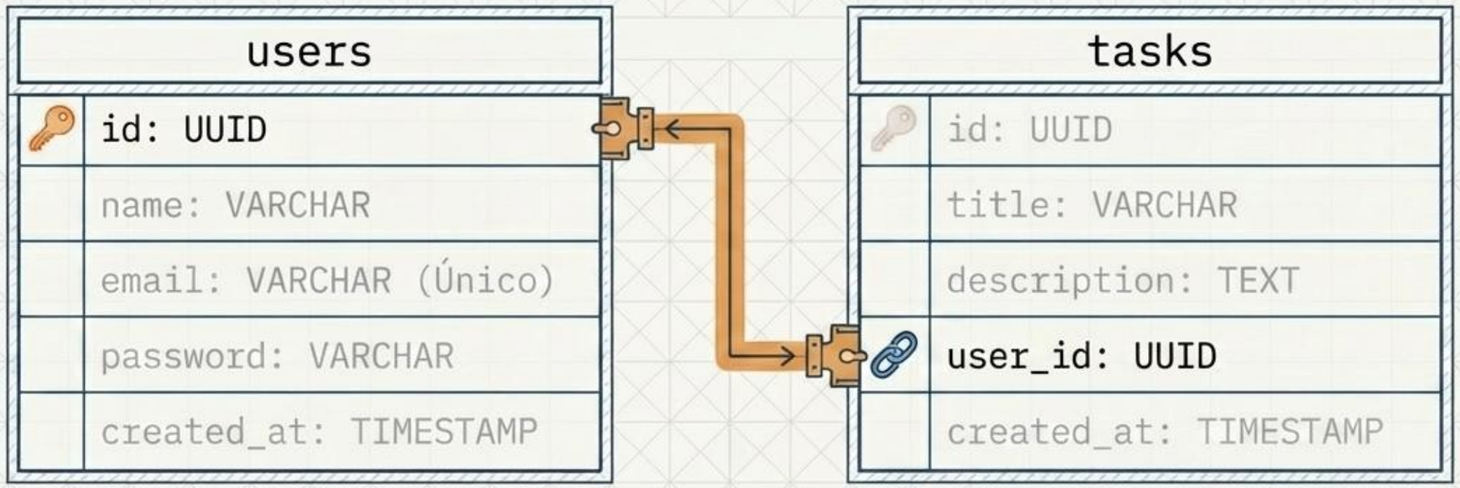
NotebookLM

O Esquema Mestre: Users & Tasks



NotebookLM

A Anatomia do Relacionamento

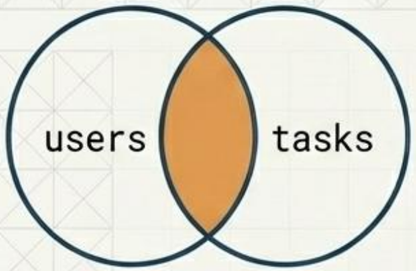


Chave Primária (PK): `users.id`. O identificador único absoluto do registro base.

Chave Estrangeira (FK): `tasks.user_id`. A referência estrutural que aponta de volta para o registro base, estabelecendo a ponte 1 para Muitos.

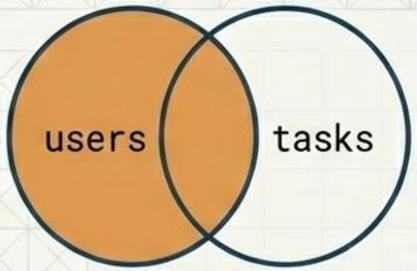
NotebookLM

A Arte dos JOINs: Conectando Tabelas



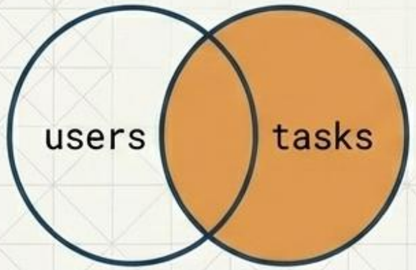
INNER JOIN

Traz apenas registros com correspondência em ambas. Quais tarefas pertencem a quais usuários?



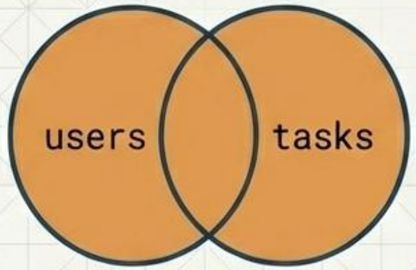
LEFT JOIN

Traz TODOS os usuários + suas tarefas (mesmo usuários sem tarefas ficam na lista com dados NULL).



RIGHT JOIN

Traz TODAS as tarefas + usuários correspondentes.

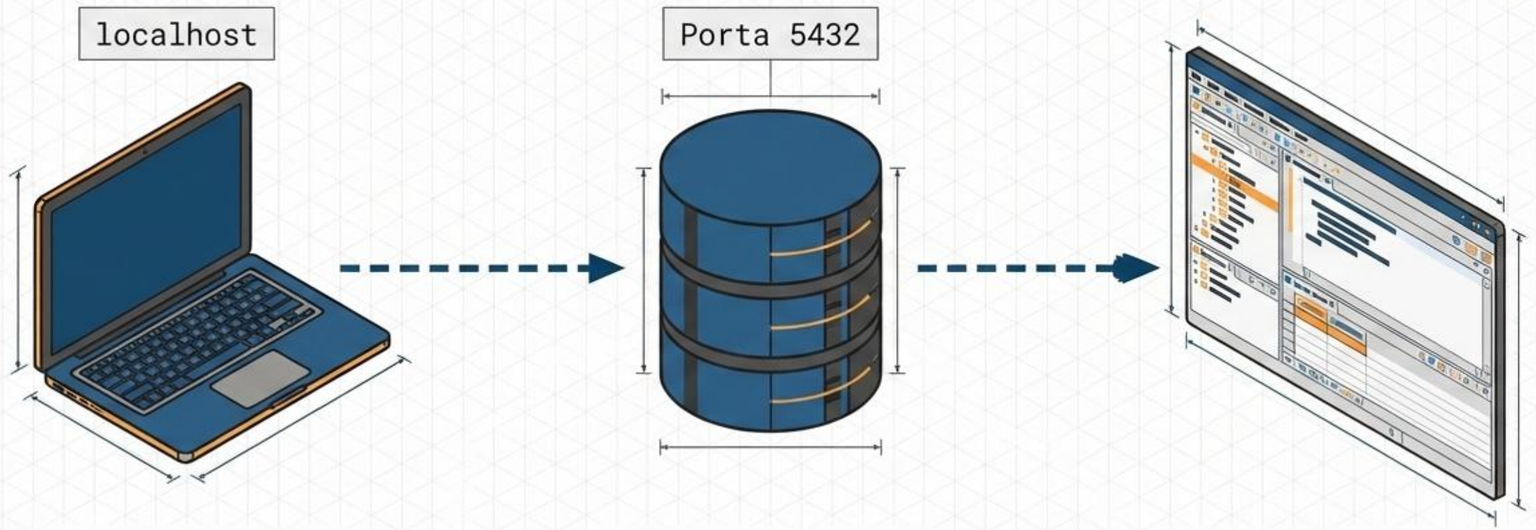


FULL OUTER JOIN

Traz todas as associações possíveis. O mapa completo.

NotebookLM

O Ecossistema do Laboratório



O ambiente local do sistema operacional (Windows/Mac/Linux).

O SGBD PostgreSQL rodando em background sob o superusuário postgres.

O software DBeaver. A interface gráfica usada para enxergar e manipular o banco task_manager_db.

NotebookLM

O Kit de Ferramentas do Engenheiro



O Motor: PostgreSQL

- Serviço invisível operando em background (porta 5432).
- Gerencia o armazenamento estrutural no disco.

O Painel de Controle: DBeaver

- A interface gráfica (GUI) gratuita e open-source.
- Permite visualizar as tabelas e executar scripts SQL visualmente.

NotebookLM


Fluxo de Fundação: Conectando e Criando
Passo 1: Host Local
Conectar ao servidor localhost:5432 usando credenciais do superusuário.
postgres
Passo 2: Rampa de Acesso
Utilizar o banco padrão (postgres) unicamente para acessar o sistema.
task_manager_db
Passo 3: Construção
Executar CREATE DATABASE task_manager_db e reconectar diretamente ao novo banco.
NotebookLM

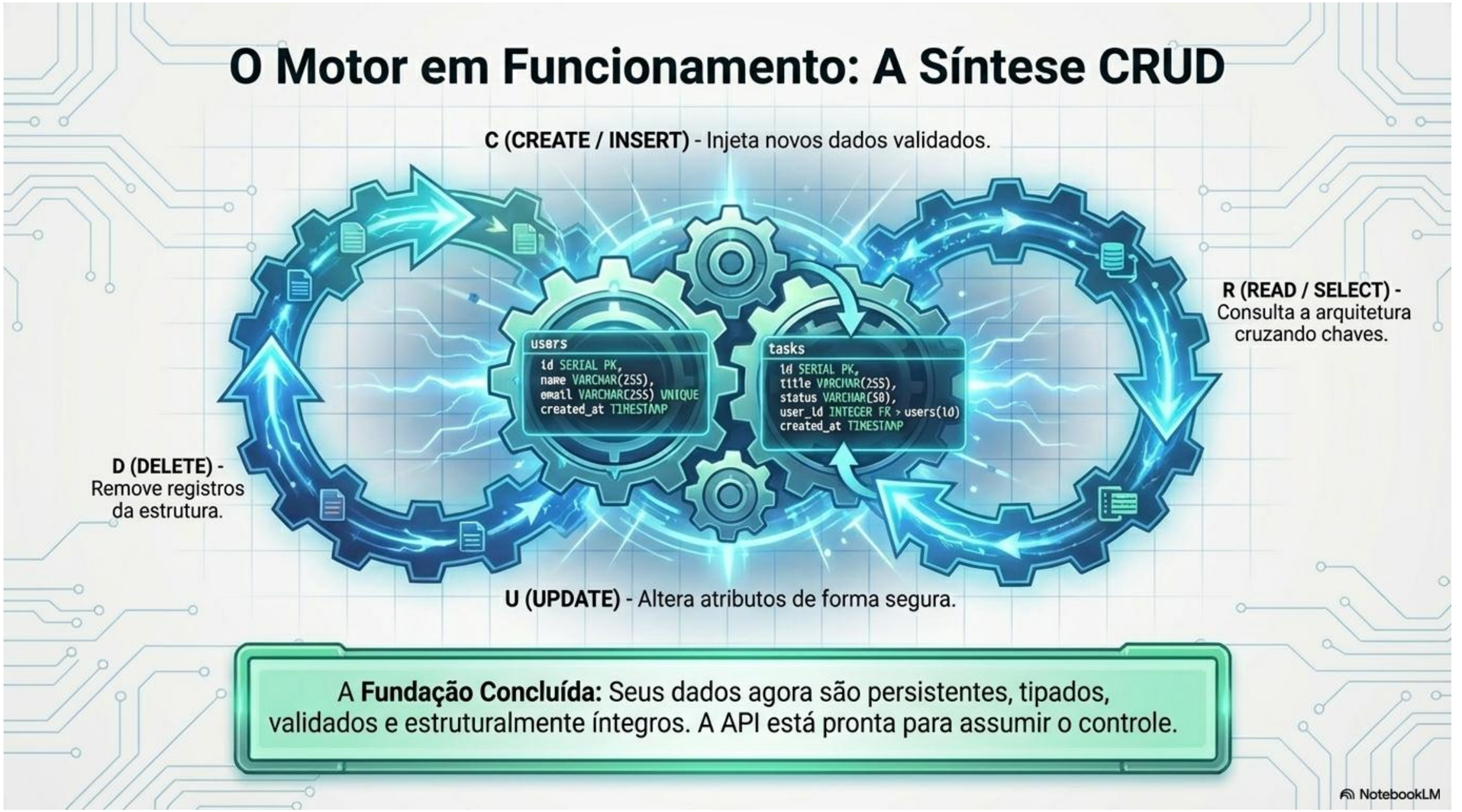
Executando o Blueprint: Materializando Tabelas

```sql
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE TABLE tasks (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  status VARCHAR(50),
  user_id INTEGER REFERENCES users(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

A sintaxe SQL transforma o design **teórico** em uma **estrutura física**. A regra **REFERENCES** bloqueia qualquer tarefa que referencie um usuário inexistente.

NotebookLM


O Motor em Funcionamento: A Síntese CRUD
C (CREATE / INSERT) - Injeta novos dados validados.
users
id SERIAL PK,
name VARCHAR(255),
email VARCHAR(255) UNIQUE
created_at TIMESTAMP
tasks
id SERIAL PK,
title VARCHAR(255),
status VARCHAR(50),
user_id INTEGER FK > users(id)
created_at TIMESTAMP
R (READ / SELECT) - Consulta a arquitetura cruzando chaves.
D (DELETE) - Remove registros da estrutura.
U (UPDATE) - Altera atributos de forma segura.
A Fundação Concluída: Seus dados agora são persistentes, tipados, validados e estruturalmente íntegros. A API está pronta para assumir o controle.
NotebookLM

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Até a Próxima Aula!

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

node JS